SOCCER GAME

El juego se define como un Arcade Sports Brawler donde la mecánica central no es el fútbol tradicional, sino el manejo de inercia y vectores de fuerza para dominar el campo de juego contra una IA progresiva.

- *Rol:* Programador de Gameplay & Debugging
- *Tecnologías:* Unity, C# (Input System Package), Physics Engine, Coroutines.

Género y Subgéneros

- *Género Principal:* Arcade Sports / Physics-based Combat (Deportes Arcade / Combate basado en físicas).
- *Subgénero:* Arena Brawler / Survival (Lucha en arena / Supervivencia por oleadas).
- *Estilo de Juego:* Physics Sandbox (Entorno de pruebas físicas), similar a títulos como Rocket League (por el fútbol con físicas) o Sumo.io (por la mecánica de empuje).

Resumen del Proyecto

Este proyecto nació como un desafío técnico de Refactorización y Debugging. El objetivo fue tomar un prototipo funcionalmente quebrado (basado en las mecánicas de Sumo Battle) y transformarlo en un juego de fútbol interactivo y estable. El jugador controla un balón mediante una cámara rotatoria, defendiendo su portería y expulsando oleadas de enemigos hacia la red contraria.

Arquitectura Técnica y Stack

- *Input Management:* Implementación de Player Input (Input Action Assets) para separar la lógica de entrada del comportamiento del jugador.
- *Physics Engine:* Uso intensivo de Rigidbody.AddForce y cálculos de vectores relativos a la cámara.

- *Control de Flujo:* Gestión de estados mediante sistemas de oleadas y corrutinas para power-ups temporales.
- *Feedback Visual:* Sistema de partículas integrado para indicar estados activos (Turbo/Power-up).

Pilares de Gameplay

- *Combate Físico:* Interacción basada en el momento y la dirección de los impactos.
- *Progresión Dinámica:* Sistema de oleadas que escala en cantidad y velocidad.
- *Navegación Inteligente:* Enemigos con IA orientada a objetivos específicos (Player Goal).

Desafíos Técnicos y Soluciones (Debugging)

Uno de los puntos fuertes de este desarrollo fue la identificación y resolución de errores críticos de lógica y física:

Desafío Detectado	Diagnóstico Técnico	Solución Implementada
Física Inversa	El vector de fuerza se calculaba hacia el jugador en el impacto.	Corrección del cálculo: Vector3 dir= (Enemigo.pos - Jugador.pos).normalized.
IA Estática	Null Reference Error por falta de referencia al objetivo.	Asignación dinámica del Player Goal mediante búsqueda de etiquetas en el Start().
Bucle de Oleadas	Generación infinita de enemigos activada por triggers incorrectos.	Refactorización del SpawnManager para validar el conteo de enemigos vivos (FindObjectsOfType).
Persistencia Power-ups		Implementación de StartCoroutine(PowerupCountdownRoutine()) para limitar el efecto a 5 segundos.

	Corrutina inactiva; el efecto no expiraba nunca.	

Gameplay y Progresión

- *Sistema de Turbo:* El jugador puede activar un impulso de velocidad (Barra Espaciadora) para maniobras defensivas rápidas.
- *Escalado de Dificultad:* Cada oleada superada incrementa la cantidad de enemigos y su velocidad base, creando una curva de aprendizaje orgánica.
- *Condiciones de Victoria/Derrota:* El juego evalúa constantemente la posición de las pelotas respecto a las áreas de portería mediante disparadores físicos (Triggers).

Controles (Configuración Input System)

- *Movimiento (WASD / Flechas):* Control vectorial relativo a la orientación de la cámara.
- *Turbo (Espacio / Botón Sur):* Impulso físico temporal.
- *Cámara:* Rotación orbital alrededor del jugador para visión periférica del campo.

Resultados Finales

- *Estabilidad:* Eliminación total de errores en la consola y comportamientos inesperados.
- *Game Feel:* Interacciones coherentes donde la fuerza del impacto depende de la velocidad relativa.
- *Pulido Visual:* Integración de sistemas de partículas para feedback inmediato al activar potenciadores.
- *Prototipo Jugable:* Una experiencia de juego completa con una curva de dificultad orgánica.